LT2 Supplementary Practice: Workshop Registration

Practice Overview

This independent exercise is optional and ungraded. It offers practice with a policy interface, controlled object state, JUnit tests and a concise design explanation. Attempt it before your assigned LT2 sitting on Wednesday, 28 October. It is a preparation exercise, not the assessed paper. After setting up the environment, you may try a 40-minute rehearsal, or work at your own pace and repeat it.

Application Context

A campus workshop accepts registration requests. Each request has a nonblank identifier, a duration of 1 to 8 whole hours, an induction-completion flag and a status. A new request is PENDING. A service consults an injected WorkshopPolicy and records either CONFIRMED or DECLINED. There is no automatic reversal of a completed decision.

Task 1: Eligibility Policy

Complete DurationPolicy.permits. A request of 1 or 2 hours is permitted whether or not induction is complete. A request of 3 or 4 hours is permitted only if induction is complete. A request of 5 to 8 hours is not permitted. The upper limits are inclusive. A null request must cause IllegalArgumentException. Checking eligibility must not change the request's status or data.

Task 2: Service Behaviour

Complete RegistrationService.confirm using the injected policy, not a newly constructed DurationPolicy. Reject a null request with IllegalArgumentException before consulting the policy. Reject an already decided request with IllegalStateException before consulting the policy, leaving its status unchanged. For a pending request, call the policy once and pass that same request. A true decision changes the status to CONFIRMED; false changes it to DECLINED. If the policy throws an exception, propagate it and leave the request PENDING. Use the supplied recordDecision method for the state change. Preserve public method names, signatures and the supplied validation.

Task 3: JUnit Tests

Write three independent @Test methods in StudentTests.java. First, confirm a two-hour request without induction and assert its CONFIRMED status. Second, decline a four-hour request without induction and assert its DECLINED status. Third, confirm a permitted request, attempt confirmation again, assert IllegalStateException and check that it remains CONFIRMED. Use a fresh object in each test. An assertThrows check needs a lambda that performs the rejected action. Add further tests if you wish, especially at 2, 4 and 5 hours and for a different policy implementation.

Task 4: Design and Testing Explanation

Complete DESIGN.md. Include a small typed class view with labelled relationships among RegistrationService, WorkshopPolicy, DurationPolicy and Registration. In no more than 80 words, explain the benefit of depending on the interface and explain one test's starting state, action and expected result. A plain text diagram is acceptable. No drawing application is required.

Development Environment and Execution

Extract the complete ZIP before opening it. In VS Code choose File > Open Folder and select EECS3311B_LT2_Optional_Practice, then Terminal > New Terminal. The terminal should be in the folder containing run-tests.sh, README.md, src, test and lib. Run java -version and javac -version. Use a full JDK 21 or later; JDK 21 matches the course setup. JUnit is bundled, so no Maven, Gradle, account or network download is required. No database, user interface or external service is needed.

On Mac or Linux run: bash run-tests.sh
On Windows PowerShell run: .\run-tests.cmd
On any supported system run: java tools/PracticeRunner.java

Before completing the TODOs, the four supplied scaffold tests should pass. They check the starter structure and validation, not the unfinished policy or service. After adding the three requested tests, you should have at least seven tests. Tests that exercise unfinished code will fail until you implement the required behaviour. Read each failure's expected and actual values. A successful run is evidence for the behaviours tested, not proof that every requirement is correct. The runner saves genuine output in results.txt and keeps earlier runs under results/. Do not edit the test output to make it appear successful. Compilation errors must be corrected before JUnit can run.

Optional Submission and Feedback

If you would like feedback, upload one ZIP to the eClass activity titled LT2 Supplementary Practice: Workshop Registration (Optional, Ungraded). Uploading is voluntary and carries no marks. There is no mandatory submission deadline for this practice. Complete it before LT2 if you wish to use it as preparation. Feedback will be provided where possible and is not guaranteed before the test.

Run java tools/PackagePractice.java YOUR_STUDENT_NUMBER, replacing YOUR_STUDENT_NUMBER with your nine-digit student number. The command creates LT2_Practice_YOUR_STUDENT_NUMBER.zip and reopens it to list its contents. Check that it contains src/, test/, tools/, DESIGN.md, results.txt, README.md and the runner scripts. Open DESIGN.md and results.txt to check your actual work is included. The library and compiled files are omitted from your submission ZIP; the original starter supplies the library for rerunning. If a command fails, read its message and keep the original files. You can also create the ZIP manually using those same contents.

In the optional eClass activity, choose Add submission, attach your ZIP and choose Save changes. Check the displayed uploaded filename. Keep your own copy. If you cannot finish, you may upload your current work with a brief note in DESIGN.md identifying where you need help. An incomplete practice submission is also welcome.

Independent Preparation

Try the work yourself first. Use your lecture materials and Java/JUnit reference documentation to clarify syntax. Be able to explain your own implementation and every assertion. The separate instructions for the assessed LT2 govern that test's permitted resources and submission arrangements.